# 國立臺南大學資訊工程學系

## 資工三「演算法」課程
## 第 4 次作業

**題目: Dictionary**

班級 ： 資工三

姓名 ： 呂建篁

學號 ： S10859045

老師：陳宗禧

中華民國 110 年 12 月 28

# Contents

**(一) 簡介及問題描述**.....1
1. 簡介.....1
2. 問題.....1
**(二) 理論分析**.....2
1. Binary Search Tree 理論.....2
2. Treap 理論.....2
3. 2-3 Tree 理論.....2
4. Hashing 理論.....2
**(三) 演算法則**.....3
1. Binary Search Tree.....3
2. Treap.....3
3. 2-3 Tree.....3
4. Hashing.....3
**(四) 程式設計環境架構**.....4
**(五) 程式 (含 source code, input code, and output code)**.....5
1. 主程式.....5
2. Input Code Format.....5
3. Output Code Format.....6
**(六) 執行結果、討論與心得**.....7
1. 執行結果.....7
2. 討論.....7
3. 心得.....7
**參考文獻資料**.....8

# (一) 簡介及問題描述

## 1. 簡介

利用 Binary Search Tree (BST), Treap (TH), 2-3 Tree (23T)and Hashing 分別建構一 Dictionary，並且利用該 Dictionary 做 Searching 以及其時間分析.

## 2. 問題

**Functions:**

i. Build **BST**, **TH**, **23T**, and **Hashing** with a Text File Input `test.txt`

a. 定義 Token & Words in Dictionary

b. 定義 **BST**, **ST**, **23T**, and **Hashing** (實作課本 295 頁(ppt 的 57-58 頁)的兩個 Hash functions, 討論不同 Hash Table Sizes, Collision 等問題)

ii. Searching with an input test file, part of the Text File `test.txt`

v. Search a word (Input from file or keyboard)

iii. Add a word (Input from file or keyboard)

iv. Delete a word (Input from file or keyboard)

# (二) 理論分析

## 1. Binary Search Tree 理論

此樹具有以下性質:

1.若任意節點的左子樹不空，則左子樹上所有節點的值均小於它的根節點的值

2.若任意節點的右子樹不空，則右子樹上所有節點的值均大於它的根節點的值

3.任意節點的左、右子樹也分別為二元搜尋樹

## 2. Treap 理論

又稱樹堆，為 Tree 及 heap 的組合，其結構類似於隨機插入的 BST，其節點須符合堆的特性，透過左右旋轉來維護其堆的特性。

## 3. 2-3 Tree 理論

23 樹可分為兩種，一種是一個資料元素(根節點)及兩個孩子(葉節點)，另一種是兩個資料元素(根節點)及三個孩子(葉節點)

若且唯若以下敘述中有一條成立時，$T$為2–3樹：

- $T$為空。即$T$不包含任何節點。
- $T$為擁有資料元素$a$的2節點。若$T$的左孩子為$L$、右孩子為$R$，則
  - $L$和$R$是等高的2–3樹；
  - $a$大於$L$中的所有資料元素；同時
  - $a$小於等於$R$中的所有資料元素。
- $T$為擁有資料元素$a$和$b$的3節點，其中$a < b$。若$T$的左孩子為$L$、中孩子為$M$、右孩子為$R$，則
  - $L$、$M$、和$R$是等高的2–3樹；
  - $a$大於$L$中的所有資料元素，並且小於等於$M$中的所有資料元素；同時
  - $b$大於$M$中的所有資料元素，並且小於等於$R$中的所有資料元素。

## 4. Hashing 理論

hash function 又稱 hash algorithm ，雜湊函式可將資料壓縮且固定資料格式，藉由打亂資料的組合來獲取雜湊值，而良好雜湊函式設計可減少碰撞發生，而雜湊的運用領域非常廣泛，最常用於加密及壓縮資料

## (三) 演算法則

註:以 worst case 為優先填寫時間複雜度

1. **Binary Search Tree**

   **time complexity: O(logn)**

   **space complexity: O(n)**

2. **Treap**

   **time complexity: O(logn)**

   **space complexity: O(n)**

3. **2-3 Tree**

   **time complexity: O(logn)**

   **space complexity: O(n)**

4. **Hashing**

   **time complexity: O(n)**

   **space complexity: O(n)**

# (四) 程式設計環境架構

## 1. 程式語言

C/C++ in MS Windows

## 2. 程式開發工具

Visual C++ 2019

## 3. 電腦硬體

CPU: Intel Core i7 8550U

RAM: 8.00GB Single-Channel

## 4. 作業系統

Windows 10 家用版 64-bit

## 5. 顯示卡

Intel UHD Graphics 620

NVIDIA GeForce GTX 1050

# (五) 程式 (含 source code, input code, and output code)

## 1. 主程式

詳情請見 Dictionary_S10859045 資料夾裡的 cpp 檔

## 2. Input Code Format

[BST、Hashing]

```
TestFile1.txt:[1]  TestFile2.txt:[2] 使用何者檔案當作字典?
請輸入1或2:
```

在選擇完畢後可以對字典內的資料進行以下基本操作

```
/-------------------------------------------------------------/
[1] 插入(Insertion)     [2] 刪除(Deletion)
[3] 搜尋(Searching)     [4] 離開(Exit)
請輸入您選擇的數字:
```

## 3. Output Code Format

[BST、Hashing]

```
Token&Words: Stories 找不到
Token&Words: The 找到
Token&Words: region 找到
Token&Words: is 找到
Token&Words: home 找到
Token&Words: to 找到
Token&Words: many 找到
Token&Words: of 找到
Token&Words: country 找到
Token&Words: Muslim 找到
Token&Words: Uyghur 找到
Token&Words: population 找到
Token&Words: and 找到
Token&Words: there 找到
Token&Words: have 找到
Token&Words: been 找到
Token&Words: allegations 找到
Token&Words: of 找到
Token&Words: forced 找到
Token&Words: labour 找到
Token&Words: and 找到
Token&Words: possibly 找到
Token&Words: genocide 找到

查詢 TestFile23.txt：成功數: 33  失敗數: 5

成功率：86.842110 %
失敗率：13.157895 %
總時間: 0.021294 秒
/-------------------------------------------------------------/
[1] 插入(Insertion)     [2] 刪除(Deletion)
[3] 搜尋(Searching)     [4] 離開(Exit)
請輸入您選擇的數字:1
欲插入的文字:hi
/-------------------------------------------------------------/
[1] 插入(Insertion)     [2] 刪除(Deletion)
[3] 搜尋(Searching)     [4] 離開(Exit)
請輸入您選擇的數字:2
欲刪除的文字:hi
/-------------------------------------------------------------/
[1] 插入(Insertion)     [2] 刪除(Deletion)
[3] 搜尋(Searching)     [4] 離開(Exit)
請輸入您選擇的數字:3
欲搜尋的文字:hi
Token&Words: hi
 找不到
/-------------------------------------------------------------/
[1] 插入(Insertion)     [2] 刪除(Deletion)
[3] 搜尋(Searching)     [4] 離開(Exit)
請輸入您選擇的數字:4
請按任意鍵繼續 . . .
```

# (六) 執行結果、討論與心得

## 1. 執行結果

執行的結果符合題目所要求的，結果如(五)Output Code Format 所示

## 2. 討論

除了 Hashing 時間之外，其他三種樹的時間複雜度均為 lgn 的時間，空間複雜度四者相同，分析完時間及空間複雜度後，個人並不建議使用 Hashing 來實作字典，因為其時間複雜度沒有比其他三種樹來的好，加上 Hashing 的查詢正確率是有一定的機率會產生錯誤的結果(意即:碰撞)，因此個人會比較傾向於使用較易上手的 BST 撰寫字典。

## 3. 心得

藉由本次演算法作業撰寫，除了能複習及加深之前所學的 Tree 知識印象外，透過實際上撰寫除了更加熟悉運用其他 Tree 的演算法，也能在日後遇到需要用到 Tree 演算法的情況下，回憶所學理論及演算分析後才決定使用何種演算法技巧來獲得最好的效率。

# 參考文獻資料

[TestFile2.txt]

[TestFile21.txt]

[TestFile22.txt]

[TestFile23.txt]

https://www.bbc.com/news/business-59769393

[BST]

https://stackoverflow.com/questions/37095190/reading-in-txt-file-c

https://www.daniweb.com/programming/softwaredevelopment/threads/372708/dictionary-using-tree

http://alrightchiu.github.io/SecondRound/binary-search-tree-searchsou-xun-zi-liao-insertxin-zeng-zi-liao.html

https://www.sanfoundry.com/cpp-program-perform-dictionary-operations-binary-search-tree/

https://github.com/debasree888/Dictionary-using-Binary-search-tree/blob/master/myDictionary.c

https://www.tutorialspoint.com/cplusplus-program-to-perform-dictionary-operations-in-a-binary-search-tree

http://see-programming.blogspot.com/2013/05/implement-dictionary-using-binary.html

https://www.softwaretestinghelp.com/binary-search-tree-in-cpp/

https://courses.engr.illinois.edu/cs225/sp2021/labs/dict/

http://www.cplusplus.com/forum/general/145509/

https://www.geeksforgeeks.org/cpp-program-read-file-word-word/

https://levelup.gitconnected.com/learning-c-working-with-text-files-91104a8fab33

https://stackoverflow.com/questions/54831746/read-from-text-file-and-make-a-dictionary-text-file

[Hashing]

https://pjchender.dev/dsa/dsa-hash-table/

https://blog.techbridge.cc/2017/01/21/simple-hash-table-intro/

https://ithelp.ithome.com.tw/articles/10208884

https://hackmd.io/@jkrvivian/HJln3jU_e?type=view

https://en.cppreference.com/w/cpp/utility/hash

https://www.geeksforgeeks.org/c-program-hashing-chaining/

https://stackoverflow.com/questions/7666509/hash-function-for-string

http://puremonkey2010.blogspot.com/2010/11/c_15.html

https://www.cs.yale.edu/homes/aspnes/pinewiki/C(2f)HashTables.html?highlight=%28CategoryAlgorithmNotes%29

https://www.tutorialspoint.com/data_structures_algorithms/hash_table_program_in_c.htm

http://alrightchiu.github.io/SecondRound/hash-tableintrojian-jie.html#dat

https://benhoyt.com/writings/hash-table-in-c/

https://medium.com/@ramyjzh/data-structures-for-dummies-hash-tables-579ddd1a4389

https://www.educba.com/hash-table-in-data-structure/

https://vimsky.com/zh-tw/examples/detail/cpp-ex-----QueryPerformanceFrequency-function.html